

Linux Class 2 - Practice Task

Practice of ls options, history, date, uname, who, mv, cp and --help

Submitted by: Shivakumar

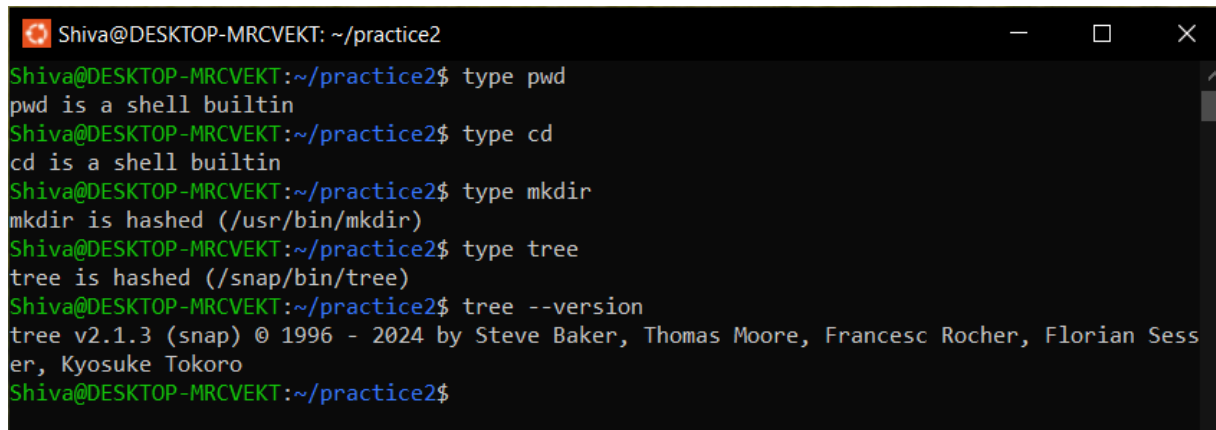
Date: 23 September 2026

Practice for class 2. I made a folder called practice2 and did all the commands from the notes there.

Internal and external commands

From the notes, internal commands are built in to Linux, like pwd and cd. External commands are the ones we have to install separately, like tree. I wanted to check this myself, so I used the type command, which tells what kind of command it is.

```
type pwd
type cd
type mkdir
type tree
tree --version
```



```
Shiva@DESKTOP-MRCVEKT: ~/practice2
Shiva@DESKTOP-MRCVEKT:~/practice2$ type pwd
pwd is a shell builtin
Shiva@DESKTOP-MRCVEKT:~/practice2$ type cd
cd is a shell builtin
Shiva@DESKTOP-MRCVEKT:~/practice2$ type mkdir
mkdir is hashed (/usr/bin/mkdir)
Shiva@DESKTOP-MRCVEKT:~/practice2$ type tree
tree is hashed (/snap/bin/tree)
Shiva@DESKTOP-MRCVEKT:~/practice2$ tree --version
tree v2.1.3 (snap) © 1996 - 2024 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sess
er, Kyosuke Tokoro
Shiva@DESKTOP-MRCVEKT:~/practice2$
```

Screenshot 1: checking which commands are built in and which are separate programs.

pwd and cd said shell builtin, so they are really inside the shell. tree is a separate program in /snap/bin/tree. tree was already installed on my system (the /snap path shows it came as a snap package), so I did not install it again. The way in the notes is sudo apt-get install tree, which asks for my password. tree --version shows it is installed.

One thing I noticed: mkdir showed /usr/bin/mkdir, so it is also a separate program. But it comes already installed with Ubuntu, we don't need to install it. I think that is why it is counted as internal in class.

sudo means super user do. It runs the command with admin (root) permission, and it asks for my password.

ls and its options

I made 3 files with echo, one by one, with a small gap, so that their times are different. I also made one hidden file. A file name starting with a dot is hidden.

```

echo "Garden city" > bangalore.txt
echo "Marina beach" > chennai.txt
echo "Howrah bridge" > kolkata.txt
touch .secret.txt
ls
ls -l
ls -lt
ls -lrt
ls -a

```

```

Shiva@DESKTOP-MRCVEKT: ~/practice2
Shiva@DESKTOP-MRCVEKT:~/practice2$ echo "Garden city" > bangalore.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ echo "Marina beach" > chennai.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ echo "Howrah bridge" > kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ touch .secret.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls
bangalore.txt  chennai.txt  kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls -l
total 12
-rw-r--r-- 1 Shiva Shiva 12 Oct  8 19:26 bangalore.txt
-rw-r--r-- 1 Shiva Shiva 13 Oct  8 19:26 chennai.txt
-rw-r--r-- 1 Shiva Shiva 14 Oct  8 19:26 kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls -lt
total 12
-rw-r--r-- 1 Shiva Shiva 14 Oct  8 19:26 kolkata.txt
-rw-r--r-- 1 Shiva Shiva 13 Oct  8 19:26 chennai.txt
-rw-r--r-- 1 Shiva Shiva 12 Oct  8 19:26 bangalore.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls -lrt
total 12
-rw-r--r-- 1 Shiva Shiva 12 Oct  8 19:26 bangalore.txt
-rw-r--r-- 1 Shiva Shiva 13 Oct  8 19:26 chennai.txt
-rw-r--r-- 1 Shiva Shiva 14 Oct  8 19:26 kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls -a
.  ..  .secret.txt  bangalore.txt  chennai.txt  kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$

```

Screenshot 2: ls, ls -l, ls -lt, ls -lrt and ls -a.

Command	What I saw
ls	Only the names
ls -l	Long format: permissions, owner, size, date and time, name
ls -lt	Long format, newest file first (kolkata.txt came first)
ls -lrt	Long format, oldest file first (bangalore.txt came first)
ls -a	All files, including hidden ones that start with a dot

Plain ls did not show .secret.txt at all. Only ls -a showed it, along with . and .. which mean the current folder and the parent folder. In ls -l the size is 12, 13 and 14 bytes, which is the number of letters in each line plus the new line.

history, date, uname, who and whoami

history | tail -5

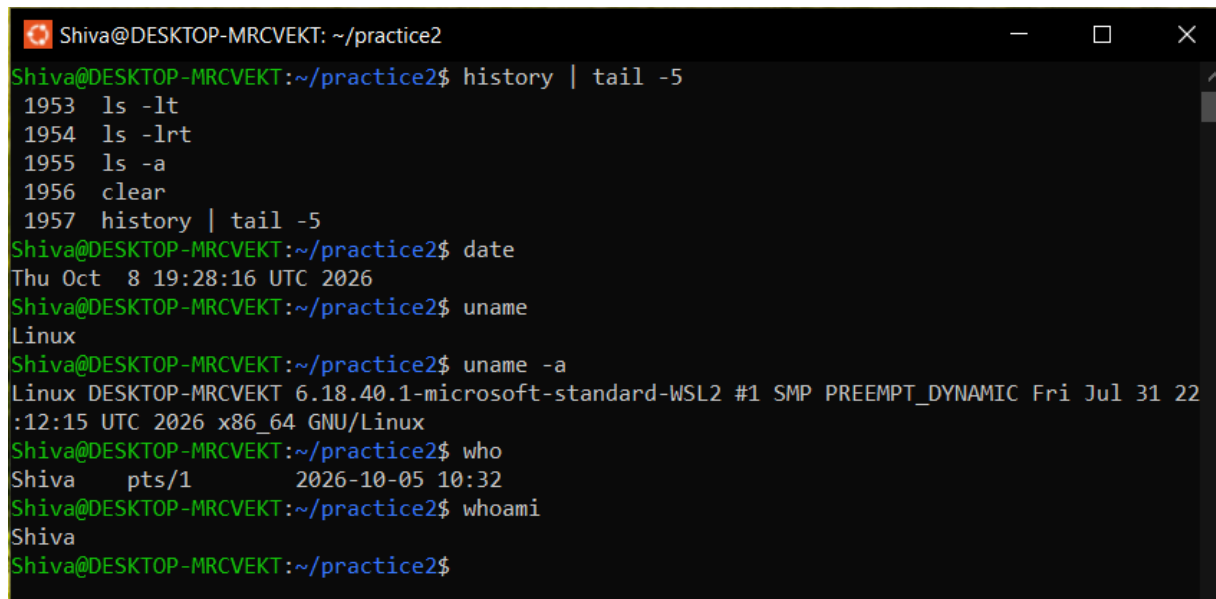
date

uname

uname -a

who

whoami



```
Shiva@DESKTOP-MRCVEKT: ~/practice2
Shiva@DESKTOP-MRCVEKT:~/practice2$ history | tail -5
1953  ls -lt
1954  ls -lrt
1955  ls -a
1956  clear
1957  history | tail -5
Shiva@DESKTOP-MRCVEKT:~/practice2$ date
Thu Oct  8 19:28:16 UTC 2026
Shiva@DESKTOP-MRCVEKT:~/practice2$ uname
Linux
Shiva@DESKTOP-MRCVEKT:~/practice2$ uname -a
Linux DESKTOP-MRCVEKT 6.18.40.1-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Fri Jul 31 22:12:15 UTC 2026 x86_64 GNU/Linux
Shiva@DESKTOP-MRCVEKT:~/practice2$ who
Shiva pts/1      2026-10-05 10:32
Shiva@DESKTOP-MRCVEKT:~/practice2$ whoami
Shiva
Shiva@DESKTOP-MRCVEKT:~/practice2$
```

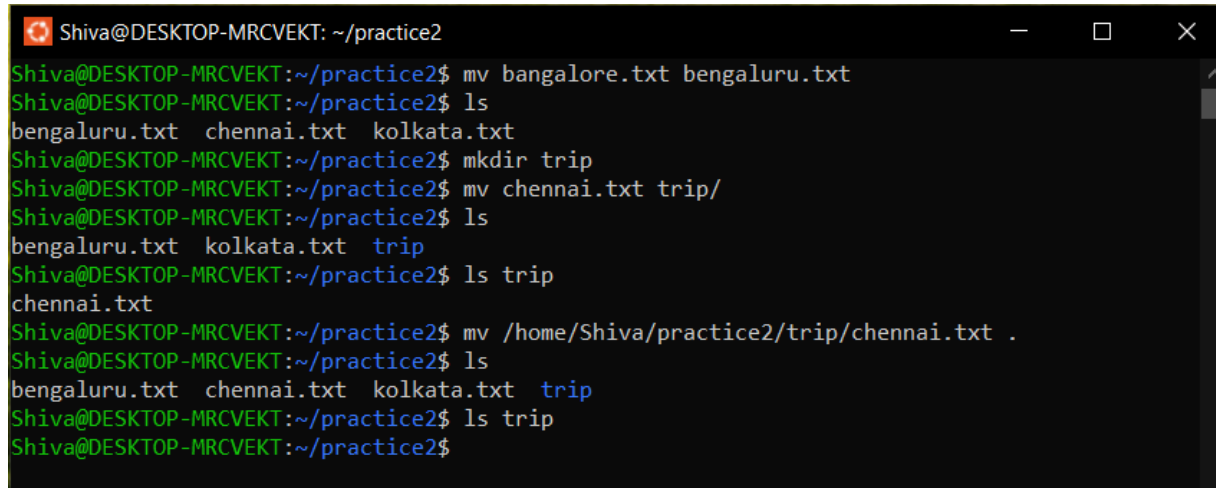
Screenshot 3: some small information commands.

- history shows all the commands I typed before. The full list was very long, so I used tail -5 to see only the last 5. Each command has a number.
- date showed the date and time of the system. It is in UTC time, not Indian time.
- uname just printed Linux, the name of the OS. In the notes it is written uname means user name, but actually it gives the system name, not my user name. My user name comes from whoami.
- uname -a gave the full details: kernel version 6.18, WSL2 (because my Ubuntu runs inside Windows) and x86_64.
- who showed my user Shiva, the terminal pts/1 and the login date and time.
- whoami printed only Shiva.

mv - rename and move

Bangalore was renamed to Bengaluru, so I did the same with my file.

```
mv bangalore.txt bengaluru.txt
mkdir trip
mv chennai.txt trip/
ls trip
mv /home/Shiva/practice2/trip/chennai.txt .
ls trip
```

A screenshot of a terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/practice2'. The terminal shows a series of commands and their outputs. The commands are: 'mv bangalore.txt bengaluru.txt', 'ls' (output: bengaluru.txt, chennai.txt, kolkata.txt), 'mkdir trip', 'mv chennai.txt trip/', 'ls' (output: bengaluru.txt, kolkata.txt, trip), 'ls trip' (output: chennai.txt), 'mv /home/Shiva/practice2/trip/chennai.txt .', 'ls' (output: bengaluru.txt, chennai.txt, kolkata.txt, trip), 'ls trip' (output: nothing), and 'ls trip' (output: nothing). The terminal uses a dark background with green and blue text for commands and outputs respectively.

```
Shiva@DESKTOP-MRCVEKT: ~/practice2
Shiva@DESKTOP-MRCVEKT:~/practice2$ mv bangalore.txt bengaluru.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls
bengaluru.txt  chennai.txt  kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ mkdir trip
Shiva@DESKTOP-MRCVEKT:~/practice2$ mv chennai.txt trip/
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls
bengaluru.txt  kolkata.txt  trip
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls trip
chennai.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ mv /home/Shiva/practice2/trip/chennai.txt .
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls
bengaluru.txt  chennai.txt  kolkata.txt  trip
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls trip
Shiva@DESKTOP-MRCVEKT:~/practice2$
```

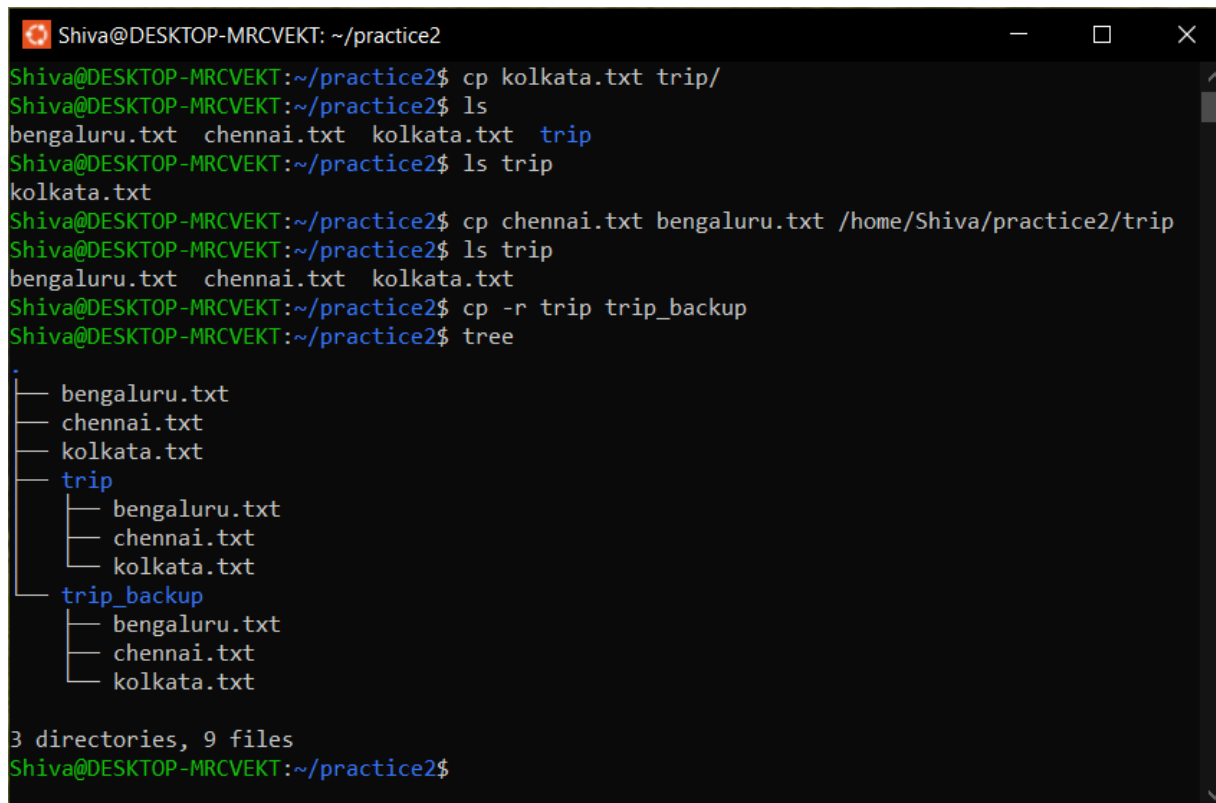
Screenshot 4: renaming a file, moving it into a folder and bringing it back.

When the second name is a new file name, mv renames the file. When the second name is a folder, mv moves the file into that folder. After moving, chennai.txt was not in practice2 anymore, it was only inside trip.

Then I moved it back using the full path, like in the notes. The dot at the end means the current folder. After this, ls trip showed nothing, so the file really moved and did not copy.

cp - copy files and folders

```
cp kolkata.txt trip/
ls trip
cp chennai.txt bengaluru.txt /home/Shiva/practice2/trip
ls trip
cp -r trip trip_backup
tree
```

A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/practice2' showing a series of commands and their outputs. The commands are: 'cp kolkata.txt trip/', 'ls', 'ls trip', 'cp chennai.txt bengaluru.txt /home/Shiva/practice2/trip', 'ls trip', 'cp -r trip trip_backup', and 'tree'. The 'tree' command output shows a directory structure with 'trip' and 'trip_backup' folders, each containing 'bengaluru.txt', 'chennai.txt', and 'kolkata.txt'. The terminal also shows '3 directories, 9 files' at the bottom.

```
Shiva@DESKTOP-MRCVEKT: ~/practice2
Shiva@DESKTOP-MRCVEKT:~/practice2$ cp kolkata.txt trip/
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls
bengaluru.txt  chennai.txt  kolkata.txt  trip
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls trip
kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ cp chennai.txt bengaluru.txt /home/Shiva/practice2/trip
Shiva@DESKTOP-MRCVEKT:~/practice2$ ls trip
bengaluru.txt  chennai.txt  kolkata.txt
Shiva@DESKTOP-MRCVEKT:~/practice2$ cp -r trip trip_backup
Shiva@DESKTOP-MRCVEKT:~/practice2$ tree
.
├── bengaluru.txt
├── chennai.txt
├── kolkata.txt
├── trip
│   ├── bengaluru.txt
│   ├── chennai.txt
│   └── kolkata.txt
└── trip_backup
    ├── bengaluru.txt
    ├── chennai.txt
    └── kolkata.txt

3 directories, 9 files
Shiva@DESKTOP-MRCVEKT:~/practice2$
```

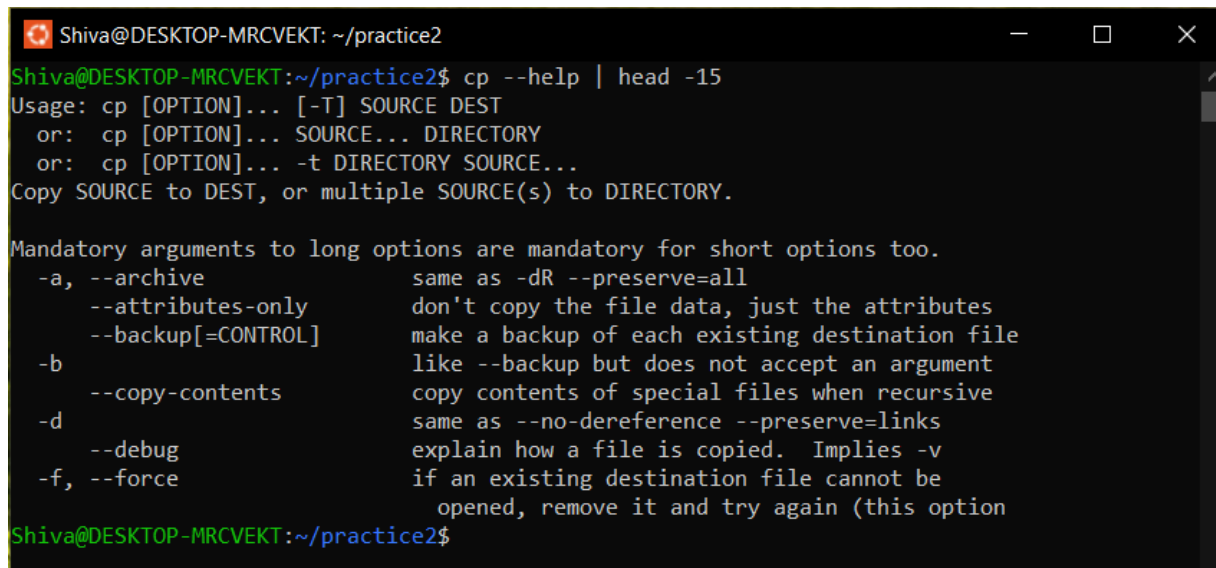
Screenshot 5: copying one file, two files and a full folder.

cp is like mv, but the original file stays where it was. After cp kolkata.txt trip/, kolkata.txt was in both places.

I can copy more than one file in one command, the last name must be the folder. For a folder I have to use cp -r, otherwise it does not copy. tree shows trip_backup has the same 3 files as trip.

--help

```
cp --help | head -15
```

A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/practice2' showing the command 'cp --help | head -15' and its output. The output displays the usage of the 'cp' command, including options like -a, -b, -d, -f, --attributes-only, --backup, --copy-contents, --debug, and --force. The terminal text is as follows:

```
Shiva@DESKTOP-MRCVEKT:~/practice2$ cp --help | head -15
Usage: cp [OPTION]... [-T] SOURCE DEST
  or:  cp [OPTION]... SOURCE... DIRECTORY
  or:  cp [OPTION]... -t DIRECTORY SOURCE...
Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY.

Mandatory arguments to long options are mandatory for short options too.
-a, --archive                same as -dR --preserve=all
--attributes-only            don't copy the file data, just the attributes
--backup[=CONTROL]          make a backup of each existing destination file
-b                           like --backup but does not accept an argument
--copy-contents              copy contents of special files when recursive
-d                           same as --no-dereference --preserve=links
--debug                     explain how a file is copied. Implies -v
-f, --force                  if an existing destination file cannot be
                             opened, remove it and try again (this option
```

Screenshot 6: the help of cp command.

--help shows how to use the command and all its options, directly in the terminal. It was very long, so I used head -15 to see only the top part. Unlike man, it does not open a separate page, it just prints and finishes.